Università Politecnica delle Marche

Ingegneria Informatica e dell'Automazione

Integrazione di una Rete Neurale Artificiale e Controllo Robotico per il Taglio Automatico di Indumenti

Corso di

Dynamics and control of intelligent robots and vehicles

Anno accademico 2025-2026

Studenti:
Gianluca Eremita
Ettore Ricci

Professore:
Andrea Bonci

Dottorando:
Alessandro Di Biase

Dipartimento di Ingegneria dell'Informazione

Indice

Introduzione **2**

1 Introduzione **3**

2 Hardware del Sistema **4**
2.1 Manipolatore UR5 4
2.2 Telecamera Intel RealSense D435i 4

3 Modulo di Visione Artificiale **6**
3.1 Dataset 6
3.2 Architettura YOLOv8-seg Nano 7
3.3 Valutazione del Modello 8
3.4 Estrazione Coordinate 10

4 Architettura ROS 2 **12**
4.1 Architettura per Unity 12
4.2 Architettura per il robot reale 12
4.3 Architettura per il robot reale - Launcher automatico 13

5 Simulazione del sistema robotico in Unity **14**
5.1 Importazione del modello del robot 14
5.2 Simulazione del capo di abbigliamento 14
5.3 Simulazione della telecamera 15
5.4 Comunicazione tra Unity e ROS 2 15
5.5 Validazione della pipeline 15
5.6 Esecuzione del sistema 16

6 Utilizzo del robot reale UR5 **19**
6.1 Polyscope e test preliminari 19
6.2 Test sul robot reale 19
6.2.1 Scelta del controller: `scaled_joint_trajectory_controller` 19
6.2.2 Utilizzo del sistema 20
6.3 Test su robot reale - Launcher automatico 21

7 Pianificazione della traiettoria cartesiana **22**
7.1 Organizzazione geometrica del percorso 22
7.1.1 Interpolazione dei punti dei polsini 23
7.1.2 Uniformazione della quota dei punti - Caso UR5 simulato 23
7.1.3 Uniformazione della quota dei punti - Caso UR5 reale 23
7.2 Costruzione dei raccordi tra i segmenti 24
7.3 Interpolazione mediante spline cubiche 25

7.4 Lunghezza della curva e parametro di avanzamento 26
7.5 Pianificazione della velocità e dell'accelerazione 26
7.5.1 Parametri cinematici dei segmenti 28
7.6 Calcolo della velocità cartesiana 28
7.7 Pianificazione dell'orientamento dell'organo terminale 29
7.7.1 Transizioni tra orientamenti 30
7.8 Concatenazione della traiettoria completa 30
7.9 Considerazioni fisiche e limiti del modello 30

8 Pianificazione Traiettorie e Controllo **32**
8.1 Pianificazione 32
8.2 Controllo UR5 32

9 Risultati e Conclusioni **33**

1 Introduzione

Il presente lavoro si colloca nell'ambito dell'automazione robotica applicata alla manipolazione e lavorazione di oggetti deformabili, con particolare riferimento al taglio automatizzato di indumenti. L'obiettivo del progetto è stato lo sviluppo di un sistema integrato di percezione e controllo in grado di riconoscere specifici punti di interesse su un maglioncino e di guidare un manipolatore robotico nell'esecuzione di un'operazione di taglio lungo traiettorie opportunamente definite.

Il sistema utilizzato è costituito da un manipolatore industriale *Universal Robots UR5*, equipaggiato con una telecamera *Intel RealSense D435i*, in grado di acquisire immagini RGB e informazioni di profondità (RGB-D). Lo sviluppo del progetto è avvenuto in una prima fase in ambiente di simulazione, utilizzando il motore grafico *Unity*, al fine di validare le strategie di percezione e controllo, e successivamente è stato esteso al sistema reale, consentendo la verifica sperimentale delle soluzioni implementate sul braccio robotico fisico.

Il task assegnato al gruppo consisteva nel riconoscimento e nella segmentazione di specifiche parti del maglioncino, in particolare polsini, fondo maglia e girocollo, mediante l'impiego di una rete neurale convoluzionale. Una volta individuate e segmentate le regioni di interesse, sono state estratte le coordinate spaziali dei punti rilevanti, utilizzate come input per la successiva fase di pianificazione della traiettoria di taglio.

All'interno del gruppo di lavoro, il progetto è stato suddiviso in due attività principali fortemente interconnesse. Una parte del gruppo si è occupata dello sviluppo del modulo di visione artificiale, comprendente la preparazione del dataset, l'addestramento della rete neurale e l'estrazione delle informazioni geometriche dalle maschere di segmentazione. L'altra parte si è concentrata sulla definizione delle traiettorie di taglio e sul controllo del manipolatore UR5, progettando una strategia di controllo in grado di seguire i percorsi generati a partire dai dati forniti dal sistema di percezione. L'integrazione tra i due moduli ha permesso di realizzare un flusso completo che va dall'acquisizione sensoriale alla realizzazione dell'azione robotica.

Lo scopo finale del lavoro è dimostrare la fattibilità di un approccio *vision-guided* per il taglio robotizzato di tessuti deformabili, evidenziando le principali sfide legate all'incertezza percettiva, alla variabilità del materiale e al controllo del manipolatore in presenza di traiettorie complesse.

La restante parte della relazione è organizzata come segue. Nel Capitolo 1 viene descritto il sistema Hardware utilizzato, con particolare attenzione al setup sperimentale. Nel Capitolo 2 viene descritto il sistema Software e l'ambiente di simulazione utilizzato. Nel Capitolo 4 viene approfondito il modulo di visione artificiale, illustrando il dataset, l'architettura della rete neurale e i risultati di segmentazione ottenuti. Nel Capitolo 5 viene presentata la pianificazione delle traiettorie e la strategia di controllo adottata per il manipolatore UR5. Infine, nel Capitolo 6 vengono discussi i risultati sperimentali e vengono tratte le conclusioni, evidenziando possibili sviluppi futuri del lavoro.

2 Hardware del Sistema

Il sistema hardware sviluppato per questo lavoro è costituito da un manipolatore collaborativo Universal Robots UR5, una telecamera RGB-D Intel RealSense D435i e una postazione di lavoro progettata per consentire la manipolazione del capo di abbigliamento in condizioni controllate. L'integrazione di questi componenti permette di acquisire le informazioni visive del maglione, elaborarle mediante algoritmi di visione artificiale e guidare il robot lungo la traiettoria di taglio pianificata.

2.1 Manipolatore UR5

Il manipolatore utilizzato è un *Universal Robots UR5*, un robot collaborativo (cobot) a sei gradi di libertà progettato per applicazioni di automazione flessibile. Grazie alla sua cinematica seriale, il robot è in grado di raggiungere un'ampia porzione dello spazio di lavoro mantenendo un'elevata precisione di posizionamento, caratteristica fondamentale per operazioni che richiedono accuratezza, come il taglio di materiali tessili.

L'UR5 presenta un raggio operativo di 850 mm e una capacità di carico nominale pari a 5 kg, caratteristiche che lo rendono adatto alla manipolazione di utensili e sensori mantenendo al contempo movimenti fluidi e sicuri. Il manipolatore è equipaggiato con sei giunti rotazionali, ciascuno controllato indipendentemente, che consentono all'end-effector di assumere qualsiasi orientamento richiesto durante l'esecuzione della traiettoria.

Il controllo del robot è realizzato mediante ROS 2 Humble utilizzando il pacchetto `ur_robot_driver`, che consente di comunicare direttamente con il controller del manipolatore. Attraverso tale interfaccia vengono acquisiti continuamente gli stati articolari del robot e inviati i riferimenti di posizione calcolati dall'algoritmo di pianificazione sviluppato nel corso del lavoro.

All'estremità del manipolatore è installata una pinza pneumatica personalizzata, progettata per ospitare un utensile di taglio costituito da una lama rotante. Tale configurazione consente al robot di seguire con precisione il profilo del maglione individuato dal sistema di visione, effettuando le operazioni di taglio lungo la traiettoria pianificata.

2.2 Telecamera Intel RealSense D435i

Per l'acquisizione delle informazioni visive è stata impiegata una telecamera Intel RealSense D435i, una camera RGB-D progettata per applicazioni di robotica, automazione industriale e visione artificiale. Il sensore consente di acquisire simultaneamente immagini a colori e mappe di profondità, fornendo per ogni pixel sia l'informazione cromatica sia la distanza dall'oggetto osservato. Questa caratteristica risulta fondamentale nelle applicazioni di manipolazione robotica, in cui è necessario ricostruire la posizione tridimensionale dei punti di interesse individuati nell'immagine.

La RealSense D435i integra una telecamera RGB ad alta risoluzione e una coppia di sensori infrarossi utilizzati per la stima della profondità mediante tecnologia a luce strutturata e stereo vision attiva. Il dispositivo è inoltre dotato di un proiettore a infrarossi che migliora

la qualità della ricostruzione tridimensionale anche in presenza di superfici poco texturizzate, come nel caso dei tessuti.

Nel sistema sviluppato, la RealSense è installata direttamente sul manipolatore mediante una configurazione *eye-in-hand*, risultando rigidamente fissata al secondo polso del robot (`wrist_2_link`). In questa configurazione il sensore segue tutti i movimenti del manipolatore, consentendo di osservare il capo di abbigliamento da una posizione sempre nota rispetto al sistema di riferimento del robot.

Le immagini RGB acquisite dalla telecamera vengono utilizzate come input della rete neurale sviluppata per l'individuazione automatica dei punti caratteristici del maglione, quali colletto, polsini e fondo. Contestualmente, la mappa di profondità permette di associare a ciascun punto rilevato una coordinata tridimensionale nel sistema di riferimento della telecamera. Attraverso la trasformazione tra il frame della telecamera e quello della base del manipolatore, ottenuta mediante la calibrazione della configurazione eye-in-hand e pubblicata nel sistema TF di ROS 2, tali punti vengono convertiti nel sistema di riferimento del robot e utilizzati per la generazione della traiettoria dell'end-effector.

3 Modulo di Visione Artificiale

Il modulo di visione artificiale ha l'obiettivo di identificare le parti salienti del maglioncino (colletto, polsini e fondo maglia) e di estrarne le coordinate spaziali necessarie per la manipolazione robotica. Il sistema si basa sull'integrazione di un modello di segmentazione ad istanze e un algoritmo di post-processing geometrico per la proiezione 3D dei punti di interesse.

3.1 Dataset

Per l'addestramento del modello è stato creato un dataset ad hoc composto da immagini RGB e mappe di profondità acquisite tramite una camera Intel RealSense D435i. Il dataset comprende immagini scattate sia in ambiente controllato (laboratorio) sia immagini provenienti da fonti online per aumentare la variabilità e la robustezza del modello.

L'operazione di etichettatura delle immagini è stata svolta manualmente mediante l'utilizzo del software **CVAT (Computer Vision Annotation Tool)**, definendo per ogni immagine i poligoni corrispondenti alle tre classi di interesse:

- **Collar**: la linea del colletto del maglioncino.
- **Cuff**: i polsini delle maniche.
- **Hem**: il bordo inferiore (fondomaglia).

I poligoni sono stati successivamente convertiti nel formato richiesto da Ultralytics YOLOv8-seg. Il dataset finale è stato suddiviso in set di training e validation (80/20) per permettere una corretta valutazione delle performance durante la fase di apprendimento.



Figure 3.1: *Immagini Presenti nel Dataset*



Figure 3.2: *Immagini Etichettate*

3.2 Architettura YOLOv8-seg Nano

La scelta del modello è ricaduta su **YOLOv8-seg Nano (YOLOv8n-seg)**, la versione più leggera e veloce della famiglia Ultralytics YOLOv8 dedicata alla segmentazione delle istanze.

Questa scelta è motivata dalla necessità di operare in tempo reale su hardware embedded, come la Jetson Orin Nano, dove le risorse computazionali e la memoria sono limitate. Nonostante le dimensioni ridotte, l'architettura YOLOv8n-seg garantisce un ottimo compromesso tra velocità (FPS) e accuratezza (mAP).

Caratteristiche principali dell'architettura:

- **Backbone**: Una versione ottimizzata di CSPDarknet53 per l'estrazione delle feature.
- **Neck**: Utilizzo di PANet (Path Aggregation Network) per migliorare la fusione delle feature a diverse scale.
- **Head**: Decoupled Head per la classificazione e la regressione dei box, integrata con un ramo dedicato alla generazione delle maschere di segmentazione tramite prototipi.

L'addestramento è stato eseguito per 30 epoche con una dimensione delle immagini di entry a 640x640 pixel, utilizzando l'ottimizzatore SGD e una funzione di loss che combina box loss, class loss e mask loss. Successivamente sono riportate immagini relative alla validazione di alcuni batch di immagini.

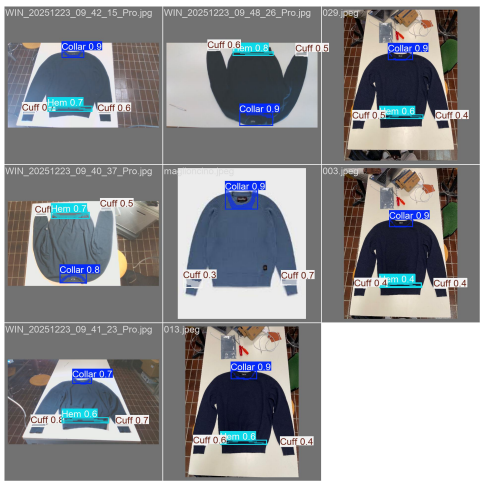


Figure 3.3: *Validazione batch 0*



Figure 3.4: *Validazione batch 1*

3.3 Valutazione del Modello

Per valutare le prestazioni della rete YOLOv8n-seg, sono state utilizzate le metriche standard per la segmentazione di istanze. In particolare, sono stati monitorati la Precision, la Recall e la media della precisione media (mAP) sia per i bounding box (rilevamento) che per le maschere (segmentazione).

I risultati ottenuti al termine dell'addestramento sono riassunti nella seguente tabella:

Metrica	**Precision**	**Recall**	**mAP@50**
Box (Detection)	85.4%	85.0%	90.3%
Mask (Segmentation)	65.4%	60.7%	56.8%

Table 3.1: *Metriche di performance del modello YOLOv8n-seg su validation set.*

L'elevato valore di mAP@50 per i box (90.3%) indica un'ottima capacità del modello di localizzare correttamente le zone di interesse. La metrica mAP@50 per le maschere (56.8%) risulta inferiore, riflettendo la complessità intrinseca della segmentazione di elementi sottili come cuciture e bordi, ma rimane comunque adeguata per l'estrazione della linea media necessaria alla manipolazione.

Successivamente sono riportati i grafici di Precision, Recall dei Box e delle Mask, insieme alla Matrice di confusione normalizzata.

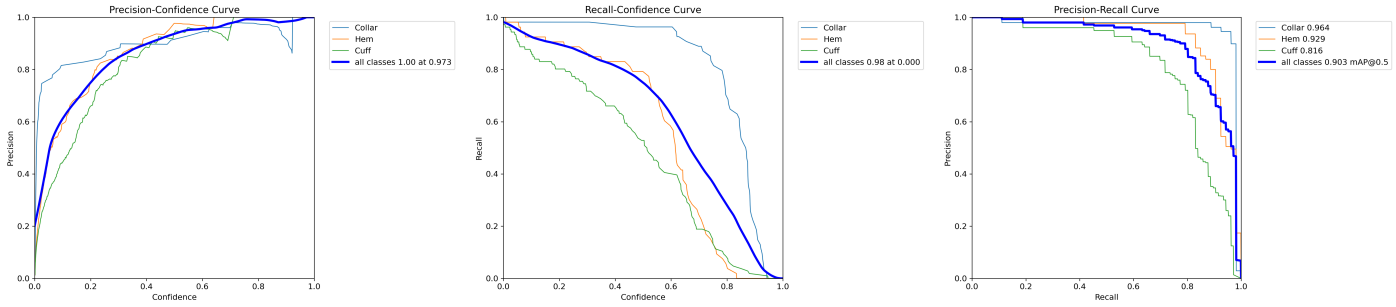


Figure 3.5: *Box Precision, Recall Confidence e Precision-Recall Curve*

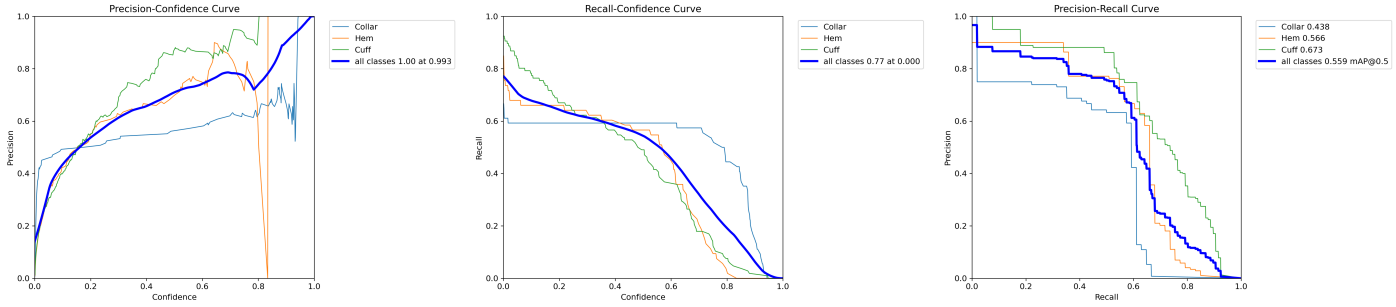


Figure 3.6: *Mask Precision, Recall Confidence e Precision-Recall Curve*

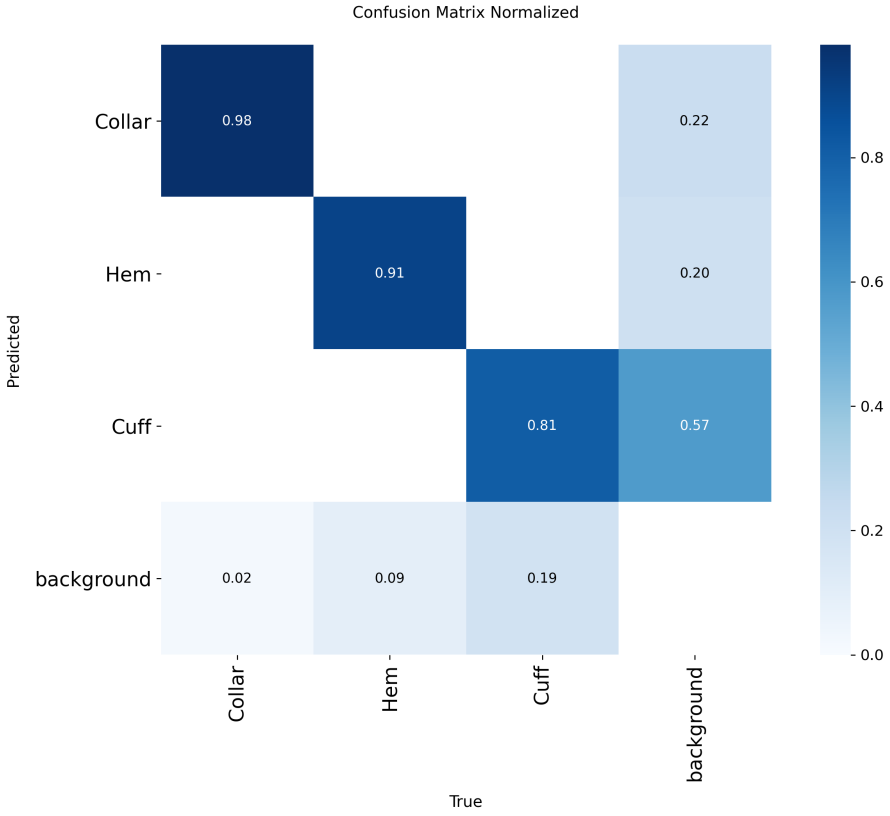


Figure 3.7: *Matrice di confusione normalizzata*

3.4 Estrazione Coordinate

Una volta ottenute le maschere dalle predizioni del modello, viene applicato un algoritmo di estrazione geometrica per individuare i punti critici.

L'algoritmo segue i seguenti passaggi:

1. **Filtraggio Maschere**: Se più istanze della stessa classe vengono rilevate, viene mantenuta solo quella con il punteggio di confidenza più elevato, applicando un filtraggio basato su Intersection over Union (IoU) per eliminare sovrapposizioni spurie.

2. **Y-Averaging (Calcolo Linea Media)**: Per ogni coordinata x lungo la maschera, viene calcolata la media dei valori y dei pixel appartenenti alla maschera. Questo permette di individuare la "linea centrale" dell'elemento (ad esempio, il centro della curva del colletto) ignorando lo spessore del bordo.

3. **Campionamento Punti**:

   - Per il **Collar** e l' **Hem**, vengono estratti N punti equidistanti lungo l'asse x per rappresentare la curvatura della cucitura.
   - Per i **Cuff**, vengono estratti unicamente i punti estremi (inizio e fine) per definire l'orientamento del polsino.

4. **Proiezione 3D**: Per ogni punto (x, y) estratto nell'immagine 2D, viene recuperato il valore di profondità z dal frame depth sincronizzato. Sfruttando i parametri intrinseci della camera, le coordinate pixel vengono infine trasformate in coordinate tridimensionali nel sistema di riferimento del robot.

Le coordinate estratte vengono salvate in formato JSON per essere utilizzate dal modulo di pianificazione delle traiettorie.

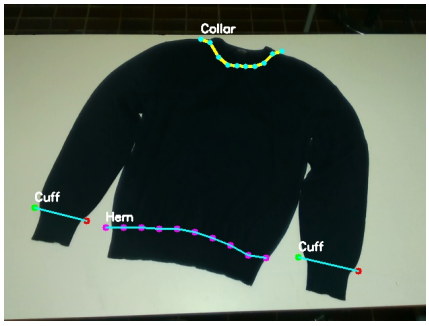


Figure 3.8: *Estrazione dei punti*

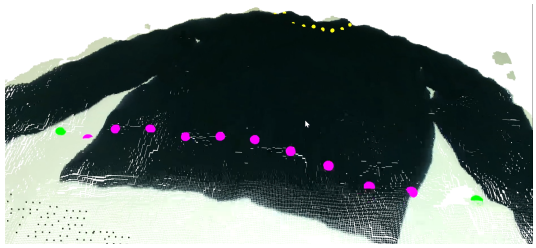


Figure 3.9: *Ricostruzione 3D dei punti estratti*

Esempio di JSON:

```
{
    "Cuff": [
        {
            "start": [
                x, y, z
            ],
            "end": [
                x, y, z
            ]
        },
        {
            "start": [
                x, y, z
            ],
            "end": [
                x, y, z
            ]
        }
    ],
    "Hem": [
        {
            "points": [
                [
                    x, y, z
                ],
                ...
            ]
        }
    ],
    "Collar": [
        {
            "points": [
                [
                    x, y, z
                ],
                ...
            ]
        }
    ]
}
```

4 Architettura ROS 2

Di seguito viene descritta l'architettura del sistema di controllo implementato in ROS 2, evidenziando i principali nodi e le loro interazioni. L'architettura è stata progettata per garantire modularità, scalabilità e facilità di integrazione sia con il sistema di simulazione in Unity che con il robot reale.

4.1 Architettura per Unity

L'architettura del sistema di simulazione in Unity è composta da diversi componenti principali, ognuno dei quali svolge un ruolo specifico nell'integrazione con ROS 2 e nella simulazione del comportamento del robot. I principali componenti includono:

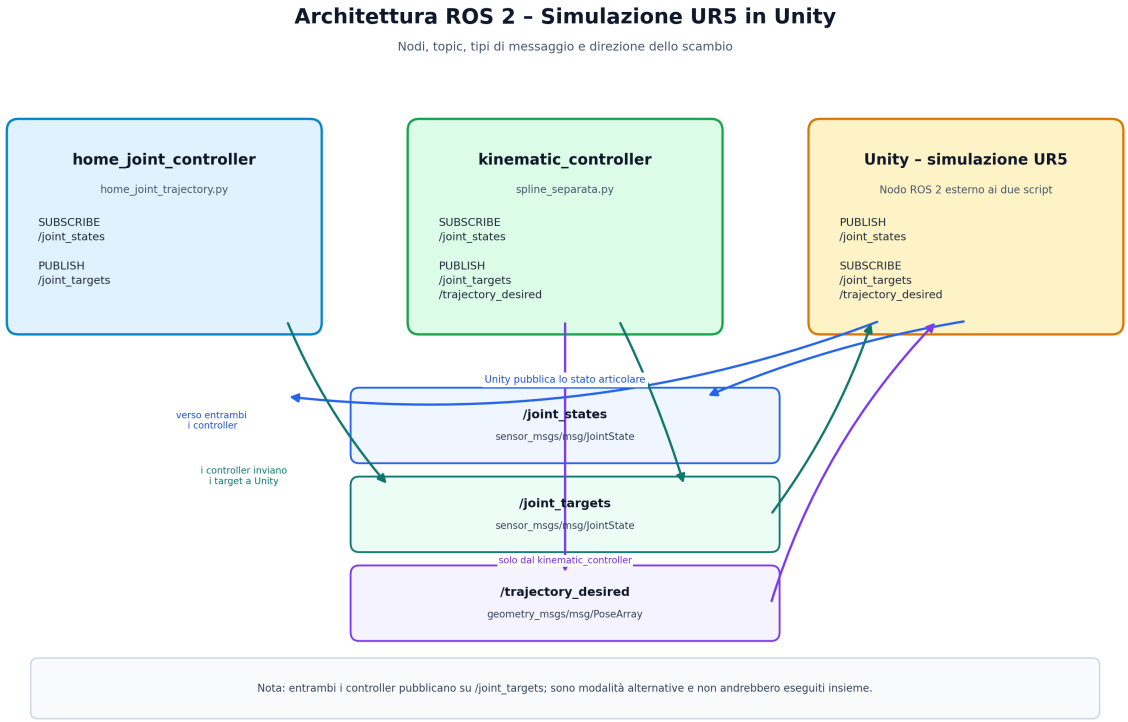


Figure 4.1: *Architettura ROS 2 per Unity*

4.2 Architettura per il robot reale

L'architettura del sistema di controllo per il robot reale si basa sulla struttura precedente ma con alcune modifiche per adattarsi alle specifiche del robot e alle sue capacità hardware. Nello specifico, invece di utilizzare messaggi su topic per la comunicazione con il robot, si fa

uso di action server e action client per gestire le richieste di movimento e le risposte del robot in modo più efficiente, coerentemente con il driver e controller dell'UR5. Rispetto al caso precedente, c'è inoltre una trasformazione statica dalla camera al `wrist_2_link` che permette di allineare correttamente le coordinate del sistema di riferimento della camera con quelle del robot tramite poi l'utilizzo di *TF*. I principali componenti includono:

4.3 Architettura per il robot reale - Launcher automatico

Questa versione dell'architettura è stata progettata per semplificare l'avvio del sistema di controllo del robot reale, automatizzando il processo di lancio dei nodi necessari e garantendo una configurazione coerente e ripetibile. Il launcher automatico consente di avviare tutti i componenti del sistema con un singolo comando, comprendendo la traiettoria per la *home_position*, lo scatto della foto, l'inferenza del modello e la traiettoria per i punti estratti. In questo modo si elimina la necessità di avviare manualmente ogni nodo e di riportare manualmente le coordinate dei punti estratti, riducendo il rischio di errori e migliorando l'efficienza complessiva del processo di controllo del robot.

5 Simulazione del sistema robotico in Unity

Prima della sperimentazione sul robot reale è stato sviluppato un ambiente di simulazione in Unity, con l'obiettivo di validare l'intera pipeline di controllo e verificare il comportamento del sistema in un ambiente virtuale. La simulazione ha permesso di testare il modello cinematico del manipolatore, la comunicazione con ROS 2, la simulazione del sensore di visione e l'interazione con un capo di abbigliamento deformabile, riducendo tempi e rischi durante la fase di sviluppo.

5.1 Importazione del modello del robot

Il manipolatore utilizzato è un Universal Robots UR5, il cui modello è stato importato in Unity a partire dalla descrizione URDF (Unified Robot Description Format). L'importazione è stata effettuata mediante il pacchetto *Unity Robotics Hub*, che consente di convertire automaticamente la struttura del robot in una gerarchia di oggetti Unity mantenendo la relazione cinematica tra i vari link.

Durante l'importazione vengono creati automaticamente tutti i collegamenti tra i link del robot, rispettando la struttura descritta nel file URDF. Ogni giunto rotazionale viene convertito in un componente *ArticulationBody*, introdotto da Unity per la simulazione fisica dei robot. Tale componente implementa un modello dinamico basato sul motore fisico PhysX e permette di simulare correttamente vincoli articolari, masse, inerzie e limiti meccanici.

I giunti del robot risultano quindi organizzati secondo la stessa catena cinematica presente sul manipolatore reale:

$$\texttt{base_link} \rightarrow \texttt{shoulder_link} \rightarrow \texttt{upper_arm_link} \rightarrow \texttt{forearm_link} \rightarrow \texttt{wrist_1_link} \rightarrow \texttt{wrist_2_link} \rightarrow \texttt{wrist_3_link}.$$

Ciò permette di utilizzare lo stesso modello cinematica impiegato nel sistema reale, garantendo una corrispondenza diretta tra simulazione ed esperimento.

5.2 Simulazione del capo di abbigliamento

Per simulare il comportamento deformabile del maglione è stato utilizzato il plugin Obi Cloth, una libreria sviluppata per Unity che implementa un modello fisico basato su particelle (Position Based Dynamics).

A differenza della fisica standard di Unity, Obi Cloth consente di rappresentare materiali tessili caratterizzati da elevate deformazioni, pieghe e interazioni con corpi rigidi. Ogni vertice della mesh del maglione viene convertito in una particella fisica collegata alle particelle vicine mediante vincoli elastici, consentendo di simulare realisticamente il comportamento del tessuto durante le deformazioni.

Il modello tridimensionale del maglione è stato importato come mesh OBJ e successivamente convertito in un oggetto Obi Cloth.

5.3 Simulazione della telecamera

Per riprodurre il comportamento della Intel RealSense D453i è stata installata una telecamera virtuale come figlia del link `wrist_2_link`, in modo che seguisse rigidamente tutti i movimenti del manipolatore.

La telecamera virtuale è stata configurata con gli stessi parametri principali della camera reale, quali campo visivo, risoluzione e posizione relativa rispetto all'end-effector.

È stato inoltre sviluppato uno script C# dedicato alla simulazione della RealSense che permette di acquisire l'immagine RGB. Le acquisizioni vengono quindi salvate sul disco in modo da poter essere elaborate successivamente dalla rete neurale utilizzata per l'estrazione dei punti caratteristici del capo.

5.4 Comunicazione tra Unity e ROS 2

L'ambiente Unity è stato integrato con ROS 2 Humble mediante il pacchetto ROS-TCP-Connector sviluppato da Unity Robotics.

Il pacchetto implementa una connessione TCP/IP tra Unity e ROS 2, consentendo lo scambio bidirezionale di messaggi in tempo reale.

Nel sistema sviluppato il nodo ROS pubblica continuamente sul topic dedicato gli angoli dei giunti desiderati del robot.

Unity sottoscrive tale topic mediante uno script C#, che converte il messaggio ROS ricevuto nei corrispondenti valori degli *ArticulationBody*. Ogni giunto viene quindi aggiornato modificando direttamente il parametro *xDrive.target*, che rappresenta la posizione angolare desiderata del giunto.

Questo approccio permette di utilizzare lo stesso nodo di controllo sviluppato per il robot reale senza apportare modifiche all'algoritmo di pianificazione della traiettoria. Dal punto di vista del software ROS, infatti, il simulatore rappresenta semplicemente un differente attuatore.

5.5 Validazione della pipeline

La simulazione ha consentito di verificare l'intera pipeline sviluppata.

In particolare è stato possibile validare:

- il modello cinematico del manipolatore;
- la corretta conversione delle coordinate tra i sistemi di riferimento della camera e del robot;
- la generazione delle traiettorie dell'end-effector;
- la sincronizzazione tra ROS 2 e Unity;
- il funzionamento della simulazione della telecamera RGB-D;
- l'interazione tra il manipolatore e il maglione.

L'utilizzo dell'ambiente virtuale ha consentito di individuare e correggere numerosi problemi relativi ai sistemi di riferimento, alle trasformazioni geometriche e alla comunicazione tra i diversi moduli software prima dell'esecuzione degli esperimenti sul robot reale, riducendo significativamente i tempi di sviluppo e aumentando l'affidabilità complessiva del sistema.

5.6 Esecuzione del sistema

Una volta stabilita la comunicazione tra Unity e ROS 2 mediante il pacchetto *ROS-TCP-Connector*, viene avviata la sequenza operativa del sistema. Il primo nodo eseguito è *home_joint.py*, il cui compito è portare il manipolatore nella configurazione di *home* precedentemente definita. Tale configurazione è stata scelta in modo da garantire che la telecamera montata sul robot inquadri completamente il maglione, assicurando condizioni di acquisizione ripetibili e una corretta visibilità delle principali caratteristiche del capo.

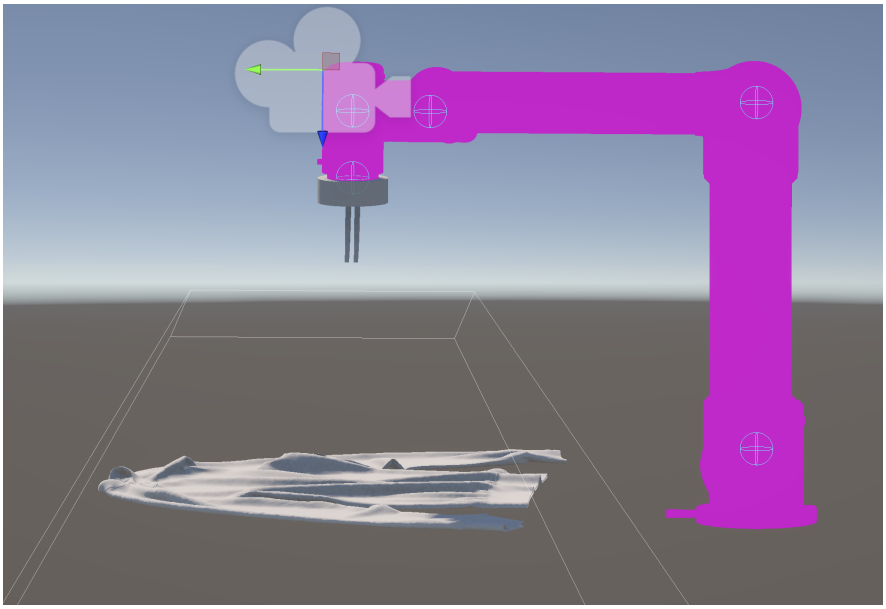


Figure 5.1: *Home Position*

Raggiunta la posizione iniziale, viene acquisita un'immagine RGB del maglione mediante la telecamera simulata. L'immagine viene salvata all'interno della cartella `Assets` del progetto Unity e successivamente utilizzata come input per la rete neurale sviluppata per l'identificazione dei punti di interesse del capo. In particolare, il modello esegue un'inferenza sull'immagine acquisita ed estrae le coordinate dei punti appartenenti al colletto, ai due polsini e al fondo del maglione, che rappresentano i riferimenti necessari per la generazione della traiettoria di manipolazione.

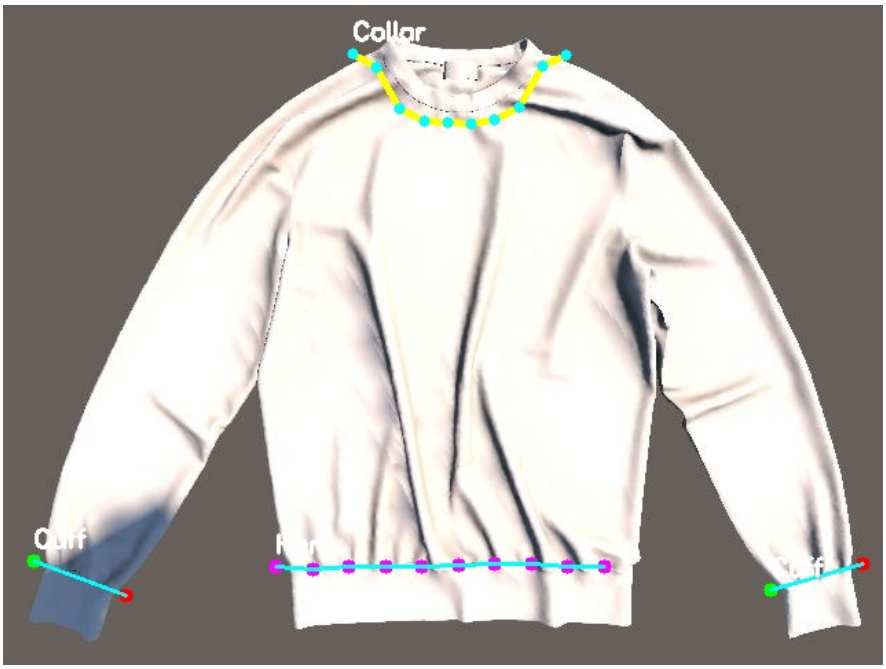


Figure 5.2: *Estrazione coordinate tramite rete neurale*

Le coordinate prodotte dalla rete neurale vengono quindi elaborate dal nodo *spline_finale.py*, che costituisce il modulo principale di pianificazione del moto. I punti vengono presi dallo script python relativi all'inferenza e successivamente inseriti manualmente all'interno nodo precedentemente menzionato. In una prima fase, i punti vengono convertiti dal sistema di riferimento della telecamera a quello della base del robot mediante opportune trasformazioni geometriche che prevedono l'utilizzo della matrice di trasformazione ricavata da Unity. Successivamente, il nodo genera una curva spline che interpola i punti estratti, ottenendo una traiettoria continua e sufficientemente regolare da poter essere seguita dal manipolatore.

Per garantire un movimento fluido e privo di brusche variazioni di velocità, la traiettoria geometrica viene parametrizzata temporalmente ed è composta da una serie di profili di velocità trapezoidali che definiscono l'evoluzione temporale del moto dell'end-effector. A ogni istante di campionamento vengono determinati il riferimento cartesiano desiderato per l'end-effector e la sua posizione attuale, ricavata dallo stato del manipolatore. Lo stato articolare corrente del manipolatore viene utilizzato per calcolare la posa effettiva dell'end-effector mediante cinematica diretta. Confrontando quest'ultima con la posa desiderata, vengono determinati l'errore di posizione e quello di orientamento. Il controllo in retroazione integra tali errori nella velocità cartesiana di riferimento, sommando al contributo di velocità pianificata un termine correttivo proporzionale all'errore. Il Jacobiano del manipolatore viene quindi invertito mediante il metodo Damped Least Squares (DLS), così da poter essere utilizzato per l'ultimo passo della cinematica inversa e ottenere i riferimenti articolari da inviare al robot attraverso i topic ROS 2.

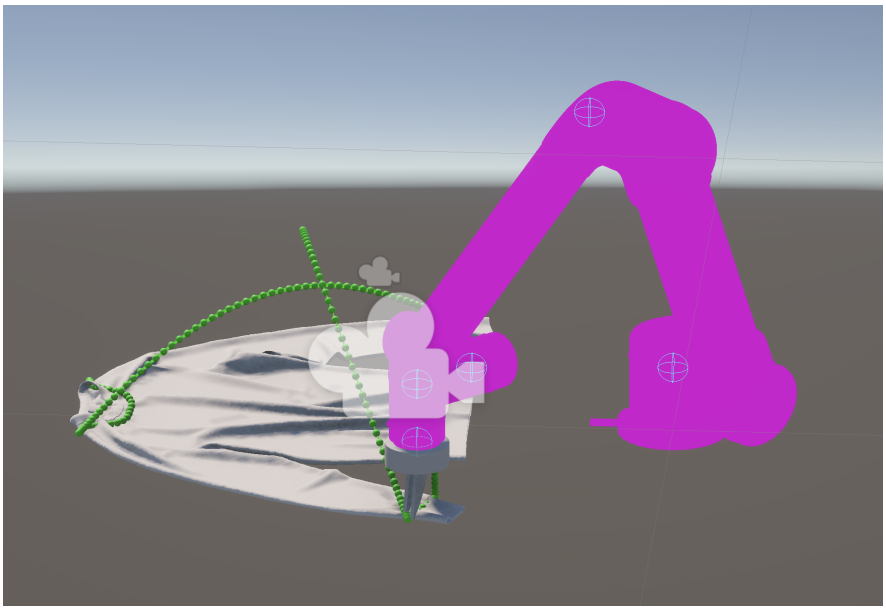


Figure 5.3: *Visualizzazione traiettoria*

La procedura viene ripetuta a ogni ciclo, aggiornando il comando sulla base dello stato corrente del robot. In questo modo, la pianificazione della traiettoria viene combinata con un controllo in retroazione che mira a ridurre gli errori di inseguimento e a migliorare la precisione del movimento. Parallelamente all'esecuzione del moto, il nodo pubblica anche l'intera traiettoria pianificata su un topic dedicato. In Unity tale traiettoria viene rappresentata graficamente mediante una sequenza di sfere distribuite lungo il percorso, consentendo una verifica immediata della correttezza della pianificazione e del movimento dell'end-effector rispetto al maglione.

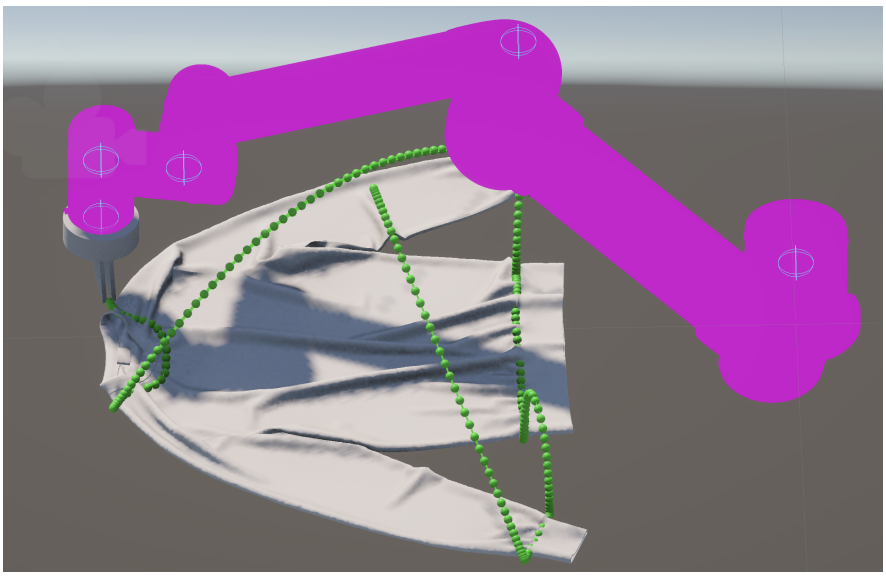


Figure 5.4: *Fine traiettoria*

6 Utilizzo del robot reale UR5

Dopo aver sviluppato e testato il sistema di controllo del robot in ambiente simulato con Unity, l'implementazione è stata estesa per consentire l'utilizzo del robot reale UR5. Questa sezione descrive le modifiche apportate all'architettura e al flusso di lavoro per adattarsi alle specifiche del robot reale, garantendo un'integrazione efficace con il sistema di controllo basato su ROS 2.

6.1 Polyscope e test preliminari

Prima ancora di passare al robot reale, abbiamo fatto utilizzo di un ulteriore simulatore, ovvero Polyscope, che ci ha permesso di testare il sistema di controllo effettivo in un ambiente virtuale più vicino alla realtà, simulando il comportamento del robot UR5 e verificando la correttezza delle traiettorie pianificate e dei comandi inviati al robot. Su Polyscope abbiamo utilizzato il driver UR5 ROS2 di default, ma un differente controller, ovvero il `forward_position_controller`, che ci ha permesso di utilizzare lo stesso metodo precedente per la trasmissione della traiettoria pianificata, ovvero la pubblicazione dei riferimenti punto per punto ad una certa frequenza.

6.2 Test sul robot reale

Dopo il processo precedente, il sistema è stato testato sul robot reale e le traiettorie pianificate sono state eseguite con successo, dimostrando l'efficacia del sistema di controllo sviluppato sebbene il metodo di pubblicazione dei riferimenti punto per punto ad una certa frequenza, anche elevata, non sia stata ottimale e presentava dei leggeri scatti durante l'esecuzione della traiettoria. Motivo per il quale, è stato deciso di passare ad un differente controller ros2 che è quello di default del driver UR5, ovvero lo `scaled_joint` e che utilizza un *action server* e un *action client* per la trasmissione della traiettoria pianificata, permettendo di inviare l'intera traiettoria in un'unica richiesta e ricevere una conferma di completamento al termine dell'esecuzione.

6.2.1 Scelta del controller: `scaled_joint_trajectory_controller`

Il driver ROS 2 del robot UR mette a disposizione diversi controller. I due più rilevanti per questo lavoro sono il `forward_position_controller` e lo `scaled_joint_trajectory_controller`.

Il `forward_position_controller` è un controller di tipo *forward command*: riceve su un topic un vettore di posizioni articolari e lo inoltra direttamente all'interfaccia hardware. Non esegue alcuna interpolazione, non applica limiti di velocità o accelerazione e non verifica il raggiungimento dell'obiettivo. Il profilo di moto dipende interamente dal nodo che pubblica i comandi, che deve quindi inviare un nuovo setpoint a frequenza costante e senza interruzioni. Un ritardo o un salto tra due setpoint successivi si traduce in un movimento brusco del robot.

Lo `scaled_joint_trajectory_controller` è invece un controller di traiettoria: riceve, tramite l'action `FollowJointTrajectory`, una sequenza di punti (posizioni, velocità e istanti

temporali) e si occupa di interpolarli e di eseguirli. Il termine *scaled* indica che il tempo della traiettoria viene scalato in base al fattore di velocità impostato sul teach pendant del robot: se l'operatore riduce la velocità o interviene una sospensione di sicurezza, la traiettoria rallenta mantenendo la propria forma, senza accumulare errore di inseguimento.

	`forward_position`	`scaled_joint_trajectory`
Ingresso	singolo setpoint (topic)	traiettoria completa (action)
Interpolazione	assente	presente
Temporizzazione	a carico del nodo	gestita dal controller
Tolleranze e esito	assenti	presenti (feedback e risultato)
Annullamento	non previsto	previsto (cancellazione del goal)
Scalatura di velocità	no	sì (teach pendant)

Table 6.1: *Confronto tra i due controller.*

Si è scelto lo `scaled_joint_trajectory_controller` per tre motivi. In primo luogo, la traiettoria viene completamente calcolata *offline* a partire dalla traiettoria cartesiana e inviata come un unico goal: il controller garantisce la temporizzazione, mentre con il `forward_position_controller` la regolarità del moto dipenderebbe dalla periodicità di un nodo Python non real-time. In secondo luogo, il controller fornisce feedback, verifica delle tolleranze e possibilità di annullare il movimento, utili per controllare l'esecuzione e fermare il robot in caso di anomalie. Infine, la scalatura del tempo rispetta i limiti e gli arresti di sicurezza imposti dal pendant, aspetto rilevante in un'applicazione in cui il robot opera in prossimità dell'operatore. Infine, grazie all'interpolazione interna, il controller garantisce un movimento più fluido e privo di scatti, migliorando la qualità complessiva dell'esecuzione della traiettoria pianificata.

6.2.2 Utilizzo del sistema

Anche in questo caso, il sistema si basa sull'utilizzo di due nodi principali: il nodo per la `home_position` e il nodo per la pianificazione ed esecuzione della traiettoria. Prima di avviare questi nodi, c'è bisogno di avviare il driver del robot UR5, e la trasformazione statica tra la camera e il `wrist_2_link` per allineare correttamente le coordinate del sistema di riferimento della camera con quelle del robot tramite l'utilizzo di *TF*. Una volta avviati questi componenti, il flusso di lavoro è simile a quello utilizzato in Unity, con la differenza che la pubblicazione ed esecuzione della traiettoria avviene tramite l'action server e l'action client del controller `scaled_joint_trajectory_controller`, garantendo un'esecuzione più fluida e sicura della traiettoria pianificata. Si parte quindi con il nodo per la `home_position`, che porta il robot in una posizione di partenza nota e sicura, e successivamente si esegue lo script python per lo scatto della foto e l'inferenza del modello, che estrae i punti di interesse dalla foto scattata dalla camera. Infine, si prendono i punti estratti e si inseriscono all'interno del nodo per la pianificazione ed esecuzione della traiettoria, che si occupa di calcolare la traiettoria ottimale per il robot e di inviarla al controller tramite l'action server.

6.3 Test su robot reale - Launcher automatico

Per semplificare ulteriormente l'utilizzo del sistema di controllo sul robot reale, è stato sviluppato un launcher automatico che avvia tutti i componenti necessari con un singolo comando. Questo launcher comprende l'avvio del nodo per la `home_position`, lo scatto della foto, l'inferenza del modello e la pianificazione ed esecuzione della traiettoria per i punti estratti. In questo modo, l'operatore può avviare l'intero processo senza doversi preoccupare di avviare manualmente ogni nodo, ma soprattutto di non dover inserire manualmente le coordinate dei punti estratti. I nodi interessati sono esattamente gli stessi utilizzati in precedenza, per cui non è richiesta una particolare modifica di essi, se non l'aggiunta di topic per notificare il componente che ha il ruolo di successivo nella catena di esecuzione. In particolare, è stata implementata la notifica dell'avvenuto completamento di una fase tramite l'utilizzo di semplici topic dedicati:

- Nodo per la `home_position` $\longrightarrow$ nodo per lo scatto della foto e l'inferenza.
- Nodo per lo scatto della foto e l'inferenza $\longrightarrow$ per la pianificazione e l'esecuzione della traiettoria.

7 Pianificazione della traiettoria cartesiana

La pianificazione della traiettoria ha lo scopo di definire il movimento dell'organo terminale del robot nello spazio cartesiano, a partire da un insieme di punti tridimensionali già individuati sulla maglia. Questi punti rappresentano i riferimenti geometrici che il robot deve seguire durante le operazioni di cucitura simulata lungo i polsini, il fondo della maglia e il colletto.

Dal punto di vista fisico, non è sufficiente collegare direttamente i punti mediante segmenti rettilinei. Una successione di segmenti può infatti introdurre discontinuità nella direzione del movimento e richiedere variazioni istantanee della velocità in corrispondenza dei vertici. Tali condizioni non sono compatibili con un movimento regolare, poiché comporterebbero accelerazioni elevate e sollecitazioni dinamiche non desiderabili.

La traiettoria viene quindi costruita attraverso tre operazioni principali:

1. organizzazione dei punti in segmenti distinti, comprendenti le cuciture e i raccordi di collegamento;
2. interpolazione geometrica mediante spline cubiche, per ottenere percorsi continui;
3. assegnazione di una legge temporale al movimento, attraverso profili di velocità e accelerazione limitati.

Il risultato è una traiettoria cartesiana descritta, per ogni istante, dalla posizione desiderata dell'organo terminale, dalla sua velocità lineare e dal suo orientamento. Grazie alla sua versatilità, il metodo realizzato può essere applicato sia al caso simulato con Unity che al caso reale con il robot UR5, con la possibilità di adattare i parametri cinematici in funzione delle esigenze operative.

7.1 Organizzazione geometrica del percorso

La funzione `genera_traiettoria_diversa()` organizza il percorso in otto segmenti consecutivi. Tale suddivisione permette di distinguere le operazioni di cucitura vere e proprie dai movimenti necessari per passare da una regione della maglia alla successiva.

Indicando con $\mathbf{P}_{\text{start}}$ la posizione iniziale dell'organo terminale e con $\mathcal{P}_{\text{sx}}$, $\mathcal{P}_{\text{fondo}}$, $\mathcal{P}_{\text{dx}}$ e $\mathcal{P}_{\text{coll}}$ gli insiemi di punti relativi, rispettivamente, al polsino sinistro, al fondo, al polsino destro e al colletto, la sequenza è organizzata come segue:

1. avvicinamento dalla posizione iniziale al primo punto del polsino sinistro;
2. percorrenza dei punti del polsino sinistro;
3. raccordo dal polsino sinistro al fondo maglia;
4. percorrenza dei punti del fondo maglia;
5. raccordo dal fondo maglia al polsino destro;

6. percorrenza dei punti del polsino destro;

7. raccordo dal polsino destro al colletto;

8. percorrenza dei punti del colletto.

Questa suddivisione consente di assegnare a ciascun tratto parametri di movimento differenti, in funzione della sua natura geometrica e delle esigenze operative.

7.1.1 Interpolazione dei punti dei polsini

Per i polsini sono inizialmente disponibili due punti estremi, che identificano l'inizio e la fine del tratto da percorrere. Per aumentare la densità dei riferimenti geometrici, vengono inseriti due punti intermedi mediante interpolazione lineare.

Indicando con $\mathbf{P}_0$ e $\mathbf{P}_1$ gli estremi del segmento, i punti intermedi sono definiti da

$$\mathbf{P}_{1/3} = \mathbf{P}_0 + \frac{1}{3}\left(\mathbf{P}_1 - \mathbf{P}_0\right), \tag{7.1}$$

$$\mathbf{P}_{2/3} = \mathbf{P}_0 + \frac{2}{3}\left(\mathbf{P}_1 - \mathbf{P}_0\right). \tag{7.2}$$

Si ottiene così una sequenza di quattro punti complessivi. L'inserimento di punti intermedi non modifica la geometria del segmento rettilineo iniziale, ma fornisce più riferimenti per la successiva interpolazione.

7.1.2 Uniformazione della quota dei punti - Caso UR5 simulato

Per la gestione delle coordinate lungo l'asse z viene utilizzata distanza fissa, corrispondente a quella tra la telecamera e il maglione, precedentemente misurata attraverso script.

7.1.3 Uniformazione della quota dei punti - Caso UR5 reale

Per ridurre le variazioni di quota tra i diversi riferimenti della maglia, la coordinata verticale dei punti dei polsini e del colletto viene uniformata utilizzando una quota di riferimento calcolata a partire dai punti del fondo.

Indicando con z_i la coordinata verticale dell'i-esimo punto del fondo, la quota comune è

$$z_{\text{ref}} = \frac{1}{N_f} \sum_{i=1}^{N_f} z_i, \tag{7.3}$$

dove N_f è il numero di punti utilizzati per descrivere il fondo.

Per i punti appartenenti ai polsini e al colletto, la coordinata verticale viene quindi sostituita con

$$z_i' = z_{\text{ref}}. \tag{7.4}$$

In questo modo, i punti di tali regioni risultano complanari rispetto alla quota scelta, mentre i punti del fondo conservano le proprie coordinate. La scelta semplifica la geometria del percorso e riduce le variazioni verticali introdotte dai punti di riferimento.

È importante precisare che questa operazione costituisce un'approssimazione geometrica: non descrive direttamente la deformazione della maglia né garantisce che la sua superficie reale sia perfettamente piana.

7.2 Costruzione dei raccordi tra i segmenti

I segmenti di cucitura sono collegati mediante tratti di transizione. Lo scopo dei raccordi è evitare che il robot debba passare direttamente da una regione della maglia a un'altra seguendo un percorso che attraversi inutilmente la superficie intermedia.

La funzione `build_trapezoids()` costruisce questi raccordi sulla base della distanza tra l'estremo del segmento precedente e l'inizio di quello successivo.

Siano $\mathbf{P}_a$ il punto finale del segmento precedente e $\mathbf{P}_b$ il punto iniziale del segmento successivo. La distanza tra i due punti, proiettati sul piano orizzontale, è calcolata come

$$d_{xy} = \sqrt{(x_b - x_a)^2 + (y_b - y_a)^2}. \tag{7.5}$$

La geometria del raccordo dipende da tale distanza:

- se $d_{xy} \geq 0.1\,\mathrm{m}$, vengono introdotti due punti intermedi;
- se $0.03\,\mathrm{m} \leq d_{xy} < 0.1\,\mathrm{m}$, viene introdotto un solo punto intermedio;
- se $d_{xy} < 0.03\,\mathrm{m}$, non vengono aggiunti punti intermedi.

Nel caso di due punti intermedi, le loro coordinate orizzontali sono ottenute dividendo lo spostamento in tre parti uguali:

$$\mathbf{p}_{1,xy} = \mathbf{p}_{a,xy} + \frac{1}{3}\left(\mathbf{p}_{b,xy} - \mathbf{p}_{a,xy}\right), \tag{7.6}$$

$$\mathbf{p}_{2,xy} = \mathbf{p}_{a,xy} + \frac{2}{3}\left(\mathbf{p}_{b,xy} - \mathbf{p}_{a,xy}\right). \tag{7.7}$$

La quota dei punti intermedi viene aumentata di una quantità prefissata:

$$z_{\mathrm{up}} = z_a + h, \qquad h = 0.1\,\mathrm{m}. \tag{7.8}$$

Quando viene inserito un solo punto intermedio, le sue coordinate orizzontali corrispondono al punto medio tra gli estremi. La quota viene calcolata con la stessa regola.

Dal punto di vista fisico, il sollevamento introduce un percorso che si allontana dalla superficie della maglia durante il trasferimento tra due regioni distanti. Questo accorgimento può ridurre il rischio di interferenza con il tessuto durante gli spostamenti, ma non costituisce una garanzia di assenza di collisioni: tale proprietà dipende anche dalla geometria effettiva della maglia, dalla posizione del tessuto e dalle dimensioni dell'organo terminale.

I raccordi vengono pertanto considerati come segmenti autonomi, ai quali è possibile assegnare limiti cinematici differenti da quelli dei tratti di cucitura.

7.3 Interpolazione mediante spline cubiche

Una volta definiti i punti di ciascun segmento, è necessario costruire una curva continua che li attraversi. A questo scopo vengono utilizzate spline cubiche naturali, calcolate separatamente per le tre coordinate cartesiane.

Sia un segmento descritto da M punti tridimensionali

$$\mathbf{P}_i = \begin{bmatrix} x_i & y_i & z_i \end{bmatrix}^T, \qquad i = 0, \ldots, M-1. \tag{7.9}$$

Per associare ai punti un parametro adimensionale crescente lungo il percorso, viene calcolata la lunghezza cumulativa della poligonale:

$$\ell_0 = 0, \tag{7.10}$$

$$\ell_i = \ell_{i-1} + \|\mathbf{P}_i - \mathbf{P}_{i-1}\|, \qquad i = 1, \ldots, M-1. \tag{7.11}$$

Il parametro viene poi normalizzato rispetto alla lunghezza totale della poligonale:

$$u_i = \frac{\ell_i}{\ell_{M-1}}, \qquad u_i \in [0, 1]. \tag{7.12}$$

La normalizzazione permette di descrivere ciascun segmento mediante un parametro che varia da zero a uno, indipendentemente dalla sua lunghezza fisica.

Vengono quindi costruite tre funzioni spline distinte:

$$x = x(u), \qquad y = y(u), \qquad z = z(u). \tag{7.13}$$

La curva cartesiana risultante è

$$\mathbf{r}(u) = \begin{bmatrix} x(u) \\ y(u) \\ z(u) \end{bmatrix}. \tag{7.14}$$

Ogni coordinata è interpolata mediante una funzione polinomiale cubica definita a tratti. Le spline naturali impongono che la derivata seconda sia nulla agli estremi del segmento:

$$x''(0) = x''(1) = 0, \tag{7.15}$$

con condizioni analoghe per $y(u)$ e $z(u)$.

La costruzione garantisce la continuità della posizione e della derivata prima all'interno del singolo segmento, evitando gli spigoli tipici della connessione diretta tra segmenti rettilinei. La derivata seconda è inoltre continua nei nodi interni della spline.

È tuttavia necessario distinguere la regolarità dei singoli segmenti dalla regolarità dell'intera traiettoria: poiché ogni tratto viene interpolato separatamente, la continuità delle derivate tra segmenti consecutivi non è automaticamente garantita dalla sola interpolazione cubica. I raccordi geometrici e le successive leggi temporali contribuiscono a rendere il movimento più regolare, ma la continuità globale deve essere valutata sui risultati ottenuti.

7.4 Lunghezza della curva e parametro di avanzamento

Il parametro u descrive la geometria della spline, ma non rappresenta direttamente la distanza percorsa. Infatti, incrementi uguali di u non corrispondono necessariamente a incrementi uguali di lunghezza lungo una curva.

Per associare al percorso una coordinata spaziale fisica, si considera la lunghezza d'arco. La lunghezza totale della spline è definita da

$$L = \int_0^1 \left\| \frac{d\mathbf{r}(u)}{du} \right\| du. \tag{7.16}$$

Nel codice, tale integrale viene approssimato numericamente campionando la curva in un elevato numero di punti e sommando le distanze euclidee tra campioni consecutivi:

$$L \simeq \sum_{i=1}^{N_s-1} \|\mathbf{r}(u_i) - \mathbf{r}(u_{i-1})\|, \tag{7.17}$$

dove N_s è il numero di campioni impiegati per l'approssimazione.

Questa grandezza, espressa in metri, rappresenta la distanza effettiva lungo la curva interpolata e costituisce il riferimento per la pianificazione della velocità.

Introducendo una coordinata di avanzamento $s(t)$, con

$$0 \leq s(t) \leq L, \tag{7.18}$$

la posizione cartesiana nel tempo può essere espressa come

$$\mathbf{x}(t) = \mathbf{r}\left(u(t)\right), \tag{7.19}$$

dove $u(t)$ è ricavato dal rapporto tra l'avanzamento lungo la curva e la lunghezza totale, mediante la relazione $u(t) = s(t)/L$.

In questo modo vengono separati due problemi distinti: la definizione della forma geometrica del percorso e la definizione della velocità con cui tale percorso viene attraversato.

7.5 Pianificazione della velocità e dell'accelerazione

La sola definizione della curva non determina quanto rapidamente il robot debba percorrerla. Per descrivere il movimento nel tempo viene quindi associato a ciascun segmento un profilo di velocità trapezoidale, o triangolare quando non è disponibile spazio sufficiente per raggiungere il valore massimo di velocità.

Il profilo è definito attraverso due parametri principali:

- $v_{\max}$, velocità lineare massima lungo la curva;
- $a_{\max}$, modulo massimo dell'accelerazione tangenziale.

La velocità scalare lungo il percorso è indicata con

$$v(t) = \frac{ds(t)}{dt}. \tag{7.20}$$

L'accelerazione tangenziale è invece

$$a_t(t) = \frac{dv(t)}{dt}. \tag{7.21}$$

Nel caso di un profilo trapezoidale, il moto è suddiviso in tre fasi:

1. accelerazione, durante la quale la velocità cresce da zero fino al valore massimo;
2. regime, durante il quale la velocità rimane costante;
3. decelerazione, durante la quale la velocità diminuisce fino a zero.

Assumendo accelerazione costante durante la prima fase, la velocità cresce secondo

$$v(t) = a_{\max} t, \tag{7.22}$$

mentre lo spazio percorso è

$$s(t) = \frac{1}{2} a_{\max} t^2. \tag{7.23}$$

Il tempo necessario a raggiungere la velocità massima è

$$t_a = \frac{v_{\max}}{a_{\max}}, \tag{7.24}$$

e la distanza percorsa durante l'accelerazione è

$$s_a = \frac{v_{\max}^2}{2a_{\max}}. \tag{7.25}$$

Una fase di decelerazione simmetrica richiede una distanza analoga. Se la lunghezza disponibile non è sufficiente a contenere entrambe le fasi e un tratto a velocità costante, il profilo diventa triangolare: la velocità massima nominale non viene raggiunta.

La funzione di pianificazione genera un profilo spaziale di velocità lungo la curva e lo converte successivamente in una successione di istanti temporali. Il tempo viene ottenuto integrando numericamente la relazione

$$dt = \frac{ds}{v(s)}. \tag{7.26}$$

Per campioni consecutivi, l'incremento temporale viene approssimato utilizzando la velocità media:

$$\Delta t_i \simeq \frac{s_i - s_{i-1}}{(v_i + v_{i-1})/2}, \tag{7.27}$$

quando la velocità media è sufficientemente diversa da zero.

Il tempo cumulativo è quindi

$$t_i = \sum_{j=1}^{i} \Delta t_j. \tag{7.28}$$

La relazione tra tempo e avanzamento viene poi interpolata per ottenere campioni temporali uniformi. Questo passaggio consente di descrivere la traiettoria attraverso una sequenza ordinata di posizioni e velocità.

7.5.1 Parametri cinematici dei segmenti

Nel codice vengono utilizzati limiti differenti in funzione del tipo di tratto. Per i segmenti ordinari di cucitura e per i raccordi non trattati come casi speciali, i valori di riferimento sono

$$v_{\max} = 0.06\,\mathrm{m/s}, \qquad a_{\max} = 0.005\,\mathrm{m/s^2}. \tag{7.29}$$

Per il segmento finale relativo al colletto vengono utilizzati valori inferiori:

$$v_{\max} = 0.03\,\mathrm{m/s}, \qquad a_{\max} = 0.002\,\mathrm{m/s^2}. \tag{7.30}$$

Per il primo segmento di avvicinamento e per il raccordo identificato dal codice con indice sei vengono invece assegnati valori più elevati:

$$v_{\max} = 0.2\,\mathrm{m/s}, \qquad a_{\max} = 0.06\,\mathrm{m/s^2}. \tag{7.31}$$

La riduzione dei limiti cinematici sul colletto è coerente con la scelta di percorrere più lentamente un tratto che richiede maggiore attenzione geometrica. Si tratta, tuttavia, di una scelta progettuale e non di un limite fisico universale: i valori devono essere verificati sperimentalmente rispetto alla precisione richiesta, alla risposta del robot e all'interazione con il tessuto.

7.6 Calcolo della velocità cartesiana

Una volta definita la legge temporale, è possibile determinare la velocità cartesiana desiderata dell'organo terminale.

Dalla relazione

$$\mathbf{x}(t) = \mathbf{r}(u(t)), \tag{7.32}$$

applicando la regola della derivazione composta si ottiene

$$\dot{\mathbf{x}}(t) = \frac{d\mathbf{r}}{du}\frac{du}{dt}. \tag{7.33}$$

La velocità cartesiana è diretta lungo la tangente alla curva. Per imporre che il suo modulo coincida con la velocità scalare pianificata $v(t)$, viene utilizzato il versore tangente

$$\mathbf{t}(u) = \frac{\mathbf{r}'(u)}{\|\mathbf{r}'(u)\|}. \tag{7.34}$$

La velocità desiderata può quindi essere scritta come

$$\dot{\mathbf{x}}_d(t) = v(t)\mathbf{t}(u(t)). \tag{7.35}$$

Esplicitando le componenti, si ottiene

$$v_x(t) = v(t)\frac{x'(u)}{\|\mathbf{r}'(u)\|}, \tag{7.36}$$

$$v_y(t) = v(t)\frac{y'(u)}{\|\mathbf{r}'(u)\|}, \tag{7.37}$$

$$v_z(t) = v(t)\frac{z'(u)}{\|\mathbf{r}'(u)\|}. \tag{7.38}$$

Questa formulazione è importante dal punto di vista fisico perché separa il modulo della velocità dalla direzione del movimento. Il profilo temporale stabilisce quanto rapidamente avanzare, mentre la derivata della spline determina in quale direzione muoversi.

La norma della velocità cartesiana risulta idealmente pari a $v(t)$:

$$\|\dot{\mathbf{x}}_d(t)\| = v(t), \tag{7.39}$$

purché la derivata della curva non sia nulla.

7.7 Pianificazione dell'orientamento dell'organo terminale

Oltre alla posizione, è necessario definire l'orientamento dell'organo terminale durante il movimento. Tale orientamento determina la direzione degli assi locali dell'utensile e contribuisce a stabilire come esso si presenta rispetto alla superficie della maglia.

Per i segmenti senza un orientamento imposto, il codice costruisce una terna cartesiana a partire dalla direzione tangente al percorso. Per ogni punto campionato, la direzione tangente viene utilizzata come asse locale x_e:

$$\mathbf{x}_e = \frac{\mathbf{r}'(u)}{\|\mathbf{r}'(u)\|}. \tag{7.40}$$

Viene quindi introdotto un vettore di riferimento verticale

$$\mathbf{z}_{\text{ref}} = \begin{bmatrix} 0 & 0 & -1 \end{bmatrix}^T. \tag{7.41}$$

Il secondo asse della terna viene calcolato mediante il prodotto vettoriale:

$$\mathbf{y}_e = \frac{\mathbf{z}_{\text{ref}} \times \mathbf{x}_e}{\|\mathbf{z}_{\text{ref}} \times \mathbf{x}_e\|}. \tag{7.42}$$

Il terzo asse viene infine ricostruito per ottenere una terna ortogonale:

$$\mathbf{z}_e = \mathbf{x}_e \times \mathbf{y}_e. \tag{7.43}$$

La matrice di orientamento desiderata è quindi

$$R_d = \begin{bmatrix} \mathbf{x}_e & \mathbf{y}_e & \mathbf{z}_e \end{bmatrix}. \tag{7.44}$$

Dal punto di vista geometrico, questa costruzione allinea il primo asse locale alla direzione di avanzamento e utilizza la verticale come riferimento per definire gli altri due assi. In questo modo l'orientamento varia in relazione alla forma della curva, anziché essere mantenuto arbitrariamente costante.

Quando il prodotto vettoriale impiegato per calcolare $\mathbf{y}_e$ ha norma molto piccola, la direzione tangente è quasi parallela al vettore verticale di riferimento. In tale condizione la

costruzione diventa numericamente mal condizionata; il codice prevede un vettore alternativo per evitare una divisione per una norma prossima a zero.

7.7.1 Transizioni tra orientamenti

Nei raccordi tra segmenti, l'orientamento iniziale e quello finale possono essere differenti. Per evitare una variazione improvvisa della posa, viene utilizzata l'interpolazione sferica dei quaternioni, nota come SLERP (*Spherical Linear Interpolation*).

Indicando con R_0 e R_1 le matrici di orientamento agli estremi del raccordo, si convertono tali orientamenti in rappresentazioni quaternioniche e si interpola tra le due pose in funzione del parametro normalizzato u:

$$R_d(u) = \mathrm{SLERP}(R_0, R_1, u). \tag{7.45}$$

La SLERP segue un percorso regolare nello spazio delle rotazioni e consente di evitare l'interpolazione indipendente degli angoli di Eulero, che può produrre comportamenti poco regolari.

La transizione dell'orientamento viene quindi coordinata con l'avanzamento lungo il raccordo. La posizione segue la spline cartesiana, mentre l'orientamento evolve gradualmente tra le due configurazioni estreme.

7.8 Concatenazione della traiettoria completa

Ogni segmento produce una sequenza temporale di posizioni, velocità cartesiane e orientamenti. Al termine della pianificazione, le sequenze vengono concatenate mantenendo l'ordine dei tratti definito in precedenza.

La traiettoria risultante può essere rappresentata come

$$\mathcal{T} = \{\mathbf{x}_d(t_i), \dot{\mathbf{x}}_d(t_i), R_d(t_i)\}_{i=0}^{N-1}, \tag{7.46}$$

dove N è il numero totale di campioni.

Per ogni istante campionato sono quindi disponibili:

- la posizione cartesiana desiderata $\mathbf{x}_d(t_i)$;
- la velocità lineare desiderata $\dot{\mathbf{x}}_d(t_i)$;
- l'orientamento desiderato $R_d(t_i)$.

La pianificazione produce così una descrizione cinematica completa del movimento nello spazio cartesiano. La traiettoria non specifica soltanto quali punti attraversare, ma anche come procedere lungo il percorso e come orientare l'organo terminale durante ciascun tratto.

7.9 Considerazioni fisiche e limiti del modello

La metodologia adottata consente di controllare separatamente la geometria del percorso e la sua evoluzione temporale. L'interpolazione con spline cubiche riduce le discontinuità

geometriche all'interno dei singoli segmenti, mentre i profili di velocità e accelerazione limitano la rapidità con cui il percorso viene attraversato.

Questi aspetti sono particolarmente rilevanti nel contesto della manipolazione di un tessuto. Un movimento troppo rapido o caratterizzato da variazioni brusche può aumentare le sollecitazioni dinamiche e compromettere la precisione con cui l'utensile segue la linea desiderata. Al contrario, una velocità più contenuta nei tratti delicati permette di adottare una strategia di movimento più prudente.

È tuttavia importante distinguere la pianificazione cinematica dalla modellazione dinamica dell'interazione con il tessuto. Il profilo di velocità limita la velocità tangenziale e l'accelerazione lungo il percorso, ma non calcola direttamente le forze di contatto, la tensione della maglia o la sua deformazione. Inoltre, l'accelerazione cartesiana totale non coincide necessariamente con la sola accelerazione tangenziale, poiché la curvatura del percorso introduce anche una componente normale:

$$\mathbf{a}(t) = a_t(t)\mathbf{t}(t) + v(t)^2\kappa(t)\mathbf{n}(t), \tag{7.47}$$

dove $\kappa(t)$ è la curvatura locale e $\mathbf{n}(t)$ è il versore normale alla traiettoria. Di conseguenza, anche quando il modulo dell'accelerazione tangenziale è limitato, curve molto strette possono generare accelerazioni normali significative.

Infine, la pianificazione viene eseguita separatamente per ciascun segmento. La continuità della posizione tra i tratti è favorita dalla costruzione dei punti di raccordo, ma la continuità della velocità e dell'accelerazione ai confini tra segmenti dipende anche dai profili temporali assegnati e dalla loro concatenazione. Per questo motivo, la verifica della traiettoria completa mediante grafici di posizione, velocità e orientamento costituisce un passaggio importante per individuare eventuali variazioni indesiderate.

In sintesi, il metodo consente di ottenere un percorso cartesiano regolare e parametrizzato nel tempo, con una velocità adattata alle diverse fasi del movimento. La qualità fisica del movimento dipende tuttavia anche dalla curvatura locale, dalla continuità tra segmenti e dall'effettiva interazione tra organo terminale e tessuto.

8 Pianificazione Traiettorie e Controllo

8.1 Pianificazione

Traiettorie generate con MoveIt! da punti segmentati. Per polsini: ellisse fitting su contour. Velocità: 50 mm/s, accelerazione 500 mm/s². Filtraggio spline per smoothing.

8.2 Controllo UR5

Controllo ibrido: posizione cartesiana con servo (impedenza 1000 N/m). Feedback visivo ogni 10 cicli per compensare deformazioni (errore ¡ 3 mm).

Pseudocodice:

```
while not trajectory_complete:
    current_pose = get_ur5_pose()
    target_pose = next_waypoint()
    delta = target_pose - current_pose
    cmd = PID(delta)
    ur5.move(cmd)
```

9 Risultati e Conclusioni

Test su 20 maglioncini: successo 85% (fallimenti per occlusioni). Tempi: segmentazione 20 ms, traiettoria 1.2 s.

Le sfide principali (deformabilità, illuminazione) sono state mitigate, dimostrando fattibilità. Futuri sviluppi: multi-view, RL per adattamento, integrazione force-torque.

Elenco delle figure

3.1 Immagini Presenti nel Dataset 6
3.2 Immagini Etichettate 6
3.3 Validazione batch 0 7
3.4 Validazione batch 1 7
3.5 Box Precision, Recall Confidence e Precision-Recall Curve 8
3.6 Mask Precision, Recall Confidence e Precision-Recall Curve 8
3.7 Matrice di confusione normalizzata 9
3.8 Estrazione dei punti 10
3.9 Ricostruzione 3D dei punti estratti 10

4.1 Architettura ROS 2 per Unity 12

5.1 Home Position 16
5.2 Estrazione coordinate tramite rete neurale 17
5.3 Visualizzazione traiettoria 18
5.4 Fine traiettoria 18